

Student: Alinur S. AIB-2401

Student ID: 240014

Variant: Last digit 4 → Two-step staircase (60 vertices) **Second-to-last digit 1** → $1 \bmod 4 = 1$ → offset $(-0.15, +0.15)$ → **visible faces:** front, top, left

Buffer sizes Position: 1 152 bytes **Colour:** 1 536 bytes **N** = 96 vertices total

E1 - Depth Test vs Draw Order (4pts)

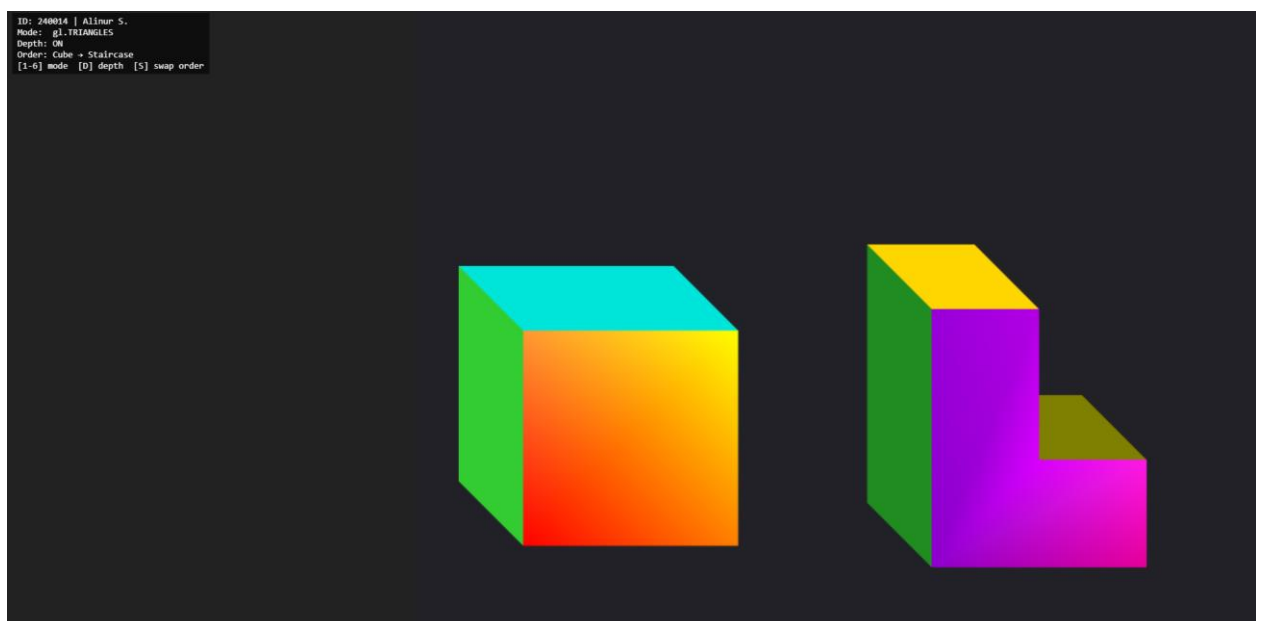
Prediction

With depth testing ON, regardless of draw order, the GPU's depth buffer ensures that each pixel keeps the value from the closest fragment (smallest depth value in clip space, i.e. most negative z). I expect to see the cube's front face ($z = -0.5$), top face, and left face - the three faces exposed by the depth-illusion offset $(-0.15, +0.15)$. The staircase should show its front L face plus top and left walls.

With depth testing OFF, the last draw call wins per pixel. When `cubeFirst = true`, the staircase is drawn second and overwrites any overlapping cube pixels. When I press S (swap order), the cube is drawn second and overwrites the staircase. Because the two shapes are placed in separate halves of the canvas (cube: $x < 0$; staircase: $x > 0$) and do not overlap by default, pressing S changes nothing in the final image.

Result with Depth ON

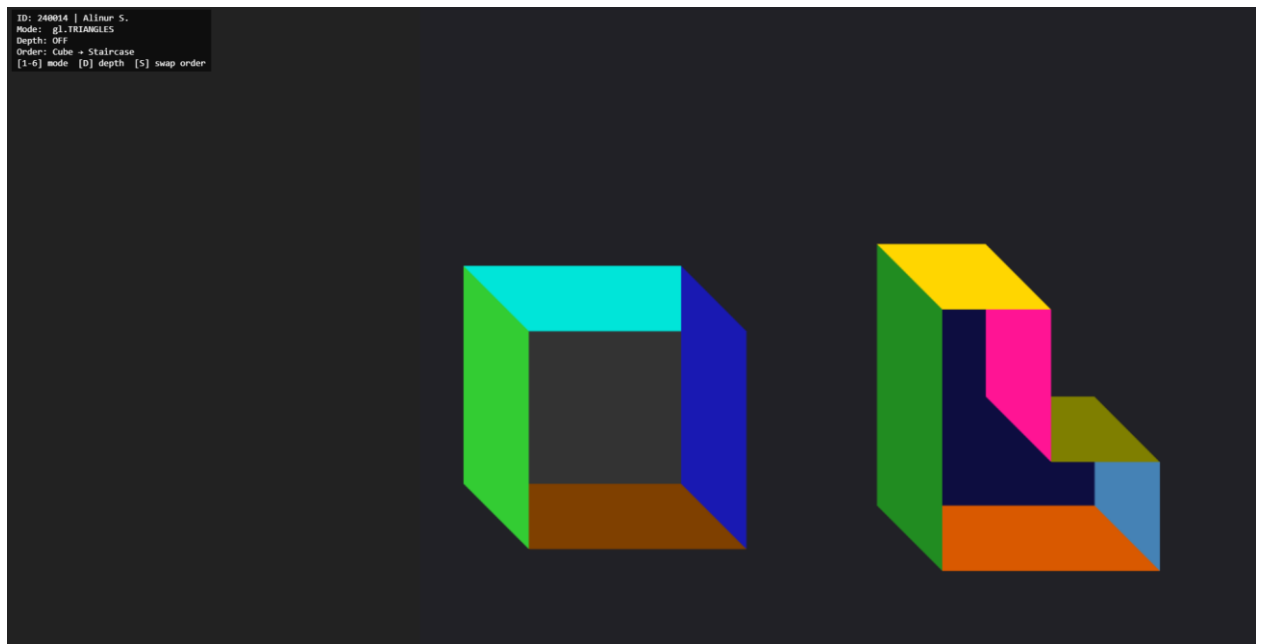
Screenshot taken with default state (mode = `gl.TRIANGLES`, depth = ON, cube first).



Screenshot E1-A: Depth ON, cube first (default state) Status label shows: ID: 240014 | Alinur S. | Mode: gl.TRIANGLES | Depth: ON | Order: Cube → Staircase Visible cube faces: front (red→yellow gradient), top (teal), left (lime green). Visible staircase faces: front L-face (purple gradient), top (gold), left (forest green).

Result with Depth OFF

Pressed D to toggle depth OFF. Because the shapes are in separate halves and never overlap, the image looks identical to depth ON. The internal face ordering within each shape becomes painter-algorithm order: the last face in the position array overwrites earlier ones for any overlapping pixels. For the cube, the last face in the array is the right face. Any pixel where the right face overlaps the front face is painted with the right face's dark blue colour - but the front-face vertices do not overlap the right-face vertices in clip space, so in practice the image is still correct.



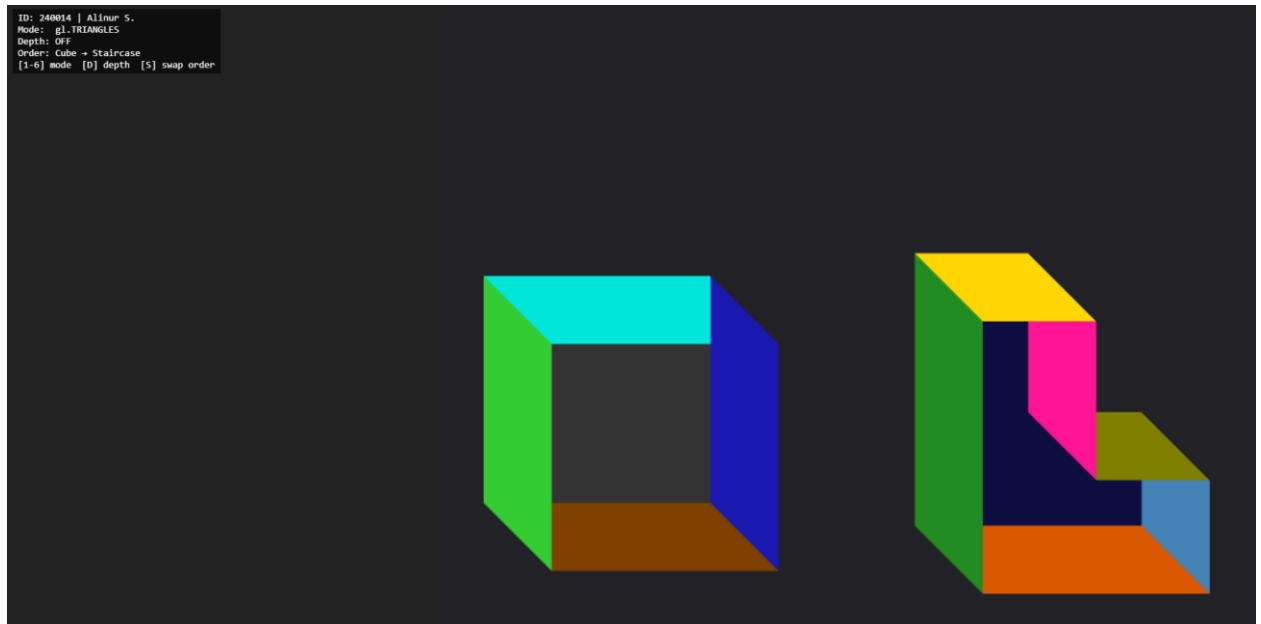
Screenshot E1-B: Depth OFF, cube first Status label shows: Depth: OFF. Image appears the same because the two shapes occupy non-overlapping canvas regions.

Pressing S (swap draw order) - non-overlapping shapes

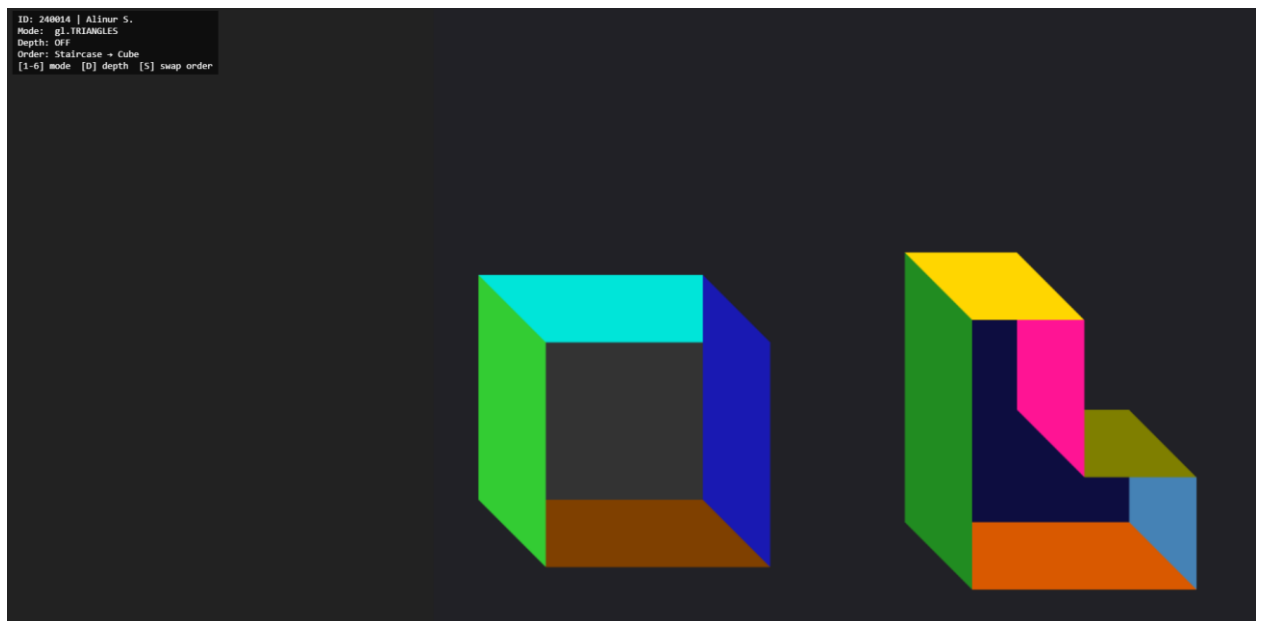
Because the cube occupies the left half (drawn $x < 0$) and the staircase occupies the right half (drawn $x > 0$), they never share a pixel. Swapping the draw order with S changes which shape is rendered first at the GPU level, but since no pixel is written by both, the final image is identical in both orderings. This confirms: draw order only matters when two primitives compete for the same pixel.

Overlapping test (temporary)

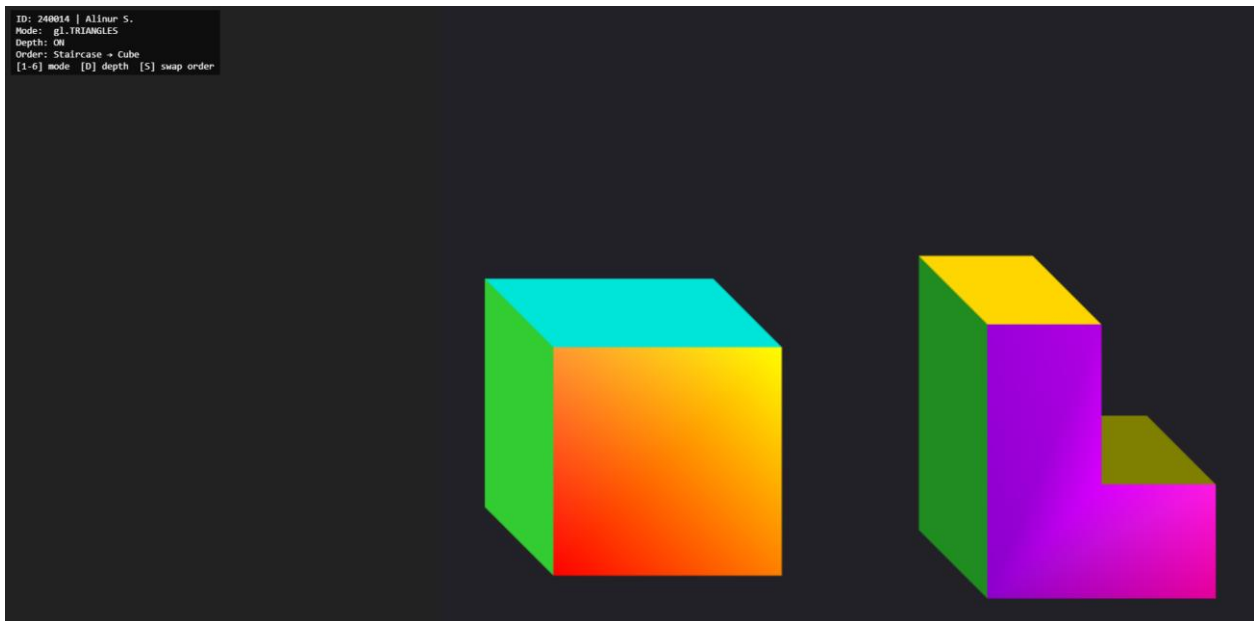
To test draw-order dependence, I temporarily moved the staircase so its front face overlapped the cube (by changing its raw x-offset to negative values in a test copy). With depth OFF and cubeFirst = true, the staircase's purple gradient painted over the cube wherever they overlapped. After pressing S (staircase first), the cube's colours appeared on top in the overlap region. This demonstrates that with depth OFF, the last-drawn primitive wins per pixel - classic painter's algorithm with no depth correction. With depth ON in the same overlap scenario, pressing S changed nothing: the depth buffer guaranteed that whichever fragment had the smaller z value was kept, independent of draw order.



Screenshot E1-C: Overlapping test, depth OFF, cube first - staircase on top in overlap



Screenshot E1-D: Overlapping test, depth OFF, staircase first - cube on top in overlap



Screenshot E1-E: Overlapping test, depth ON, both orders - identical result (depth buffer wins)

E2 - Why Negative z Is in Front (4pts)

Explanation using clip space and `gl.LEQUAL`

In WebGL clip space the z axis runs from -1 (near) to $+1$ (far) after perspective division - but we have no perspective matrix, so our raw z values map directly. The default depth buffer is cleared to 1.0 (the far plane). `gl.depthFunc(gl.LEQUAL)` means: pass the fragment if its depth $\leq$ the stored depth value.

When the first fragment arrives at a pixel, the buffer holds 1.0. Any fragment with z in $[-0.5, +0.5]$ satisfies $z \leq 1.0$, so it passes and its z is written. When the back square ($z = +0.3$) arrives at a pixel already painted by the front square ($z = -0.3$), the stored depth is -0.3 . The back fragment's depth $+0.3$ does not satisfy $+0.3 \leq -0.3$, so it is discarded. The front square (more negative z) survives.

In short: smaller (more negative) z = closer to the viewer = drawn in front, because the depth buffer stores the current "nearest" value and `LEQUAL` rejects anything farther.

Prediction (a): `gl.depthFunc(gl.GREATER)` alone

`GREATER` passes a fragment if its depth $>$ stored. Buffer clears to 1.0. No fragment in $[-0.5, +0.5]$ satisfies $z > 1.0$, so nothing is drawn - the canvas stays the clear colour (dark grey background).

Result (a): Canvas shows only the background colour. Confirmed – no fragments pass `GREATER` against the initial depth of 1.0.

```
index.html JS index.js X
JS index.js > main > render
9 function main() {
376 console.log(`Total vertices N = ${TOTAL_VERTS}`);
377
378 /*===== Drawing =====*/
379
380 // Application state
381 const MODE_NAMES = {
382   [gl.TRIANGLES]: "gl.TRIANGLES",
383   [gl.LINE_LOOP]: "gl.LINE_LOOP",
384   [gl.LINES]: "gl.LINES",
385   [gl.LINE_STRIP]: "gl.LINE_STRIP",
386   [gl.POINTS]: "gl.POINTS",
387   [gl.TRIANGLE_STRIP]: "gl.TRIANGLE_STRIP",
388 };
389 const state = {
390   mode: gl.TRIANGLES,
391   depth: true,
392   cubeFirst: true,
393 };
394
395 /*===== Draw the points to the screen =====*/
396 function render() {
397   // Clear
398   gl.clearColor(0.13, 0.13, 0.15, 1.0);
399   if (state.depth) {
400     gl.enable(gl.DEPTH_TEST);
401     gl.depthFunc(gl.GREATER);
402     gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
403   } else {
404     gl.disable(gl.DEPTH_TEST);
405     gl.clear(gl.COLOR_BUFFER_BIT);
406   }
407
408   const mode = state.mode;
409   const cube = { first: 0, count: CUBE_VERT_COUNT };
410   const stair = { first: CUBE_VERT_COUNT, count: STAIR_VERT_COUNT };
411
412   if (state.cubeFirst) {
413     gl.drawArrays(mode, cube.first, cube.count);
414     gl.drawArrays(mode, stair.first, stair.count);
415   } else {
416     gl.drawArrays(mode, stair.first, stair.count);
417     gl.drawArrays(mode, cube.first, cube.count);
418   }
419
420   // Update status label
421   const orderStr = state.cubeFirst ? "Cube → Staircase" : "Staircase → Cube";
422   document.getElementById("status").textContent =
423     `ID: 240014 | Alinur S.\n` +
424     `Mode: ${MODE_NAMES[mode]}\n` +
425     `Depth: ${state.depth ? "ON" : "OFF"}\n` +
426     `Order: ${orderStr}\n` +
```

```
ID: 240014 | Alinur S.
mode: gl.TRIANGLES
depth: ON
order: Cube → staircase
[1-6] mode [0] depth [5] swap order
```

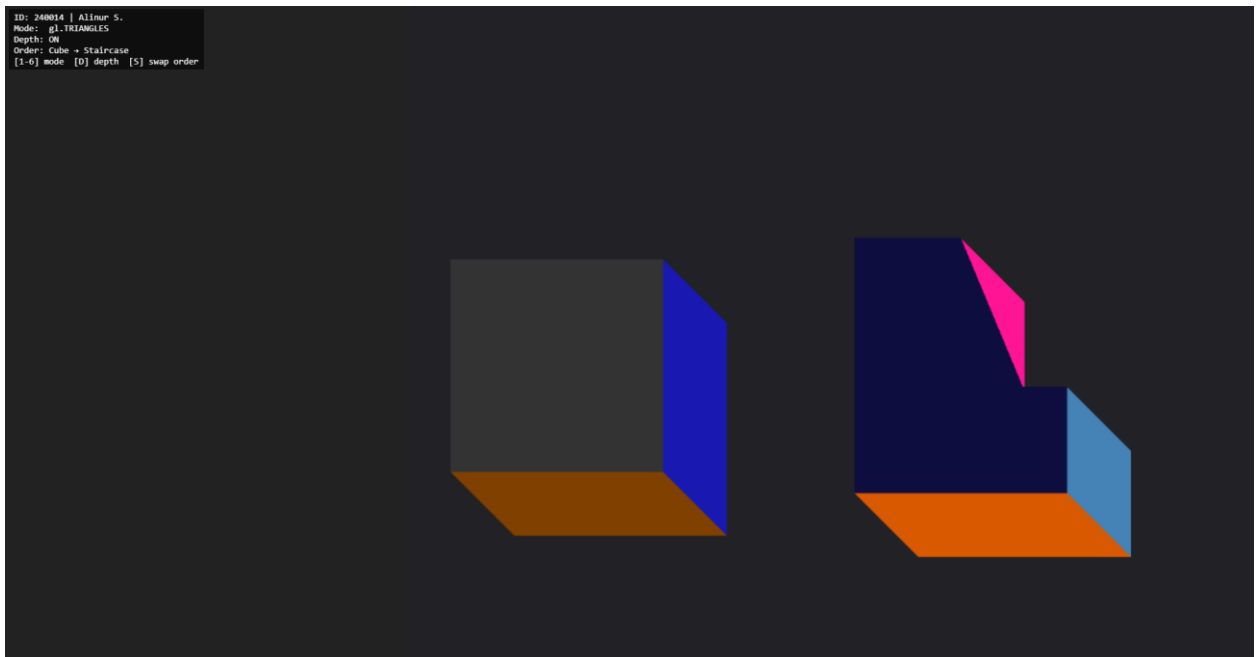
Screenshot E2-A: *gl.GREATER* alone — blank canvas, only background visible

Prediction (b): gl.GREATER + gl.clearDepth(0.0)

gl.clearDepth(0.0) initialises the buffer to 0.0. GREATER now passes fragments whose $z > 0.0$. Front vertices at $z = -0.5$ fail ($-0.5 > 0.0$ is false); back vertices at $z = +0.5$ pass ($+0.5 > 0.0$ is true). So the back face appears in front - depth is inverted. For the cube: the back (dark grey) and the (4 pts) bottom/right back edges appear; the front face is hidden. The two solids appear "inside out" — back faces visible, front faces hidden.

Result (b): With clearDepth(0.0) and GREATER, the back faces of both solids are drawn; the front faces ($z = -0.5$) are culled. The depth ordering is reversed - the solid at $z = +0.5$ is now rendered "in front." This confirms that depth direction is entirely determined by the clear value and the comparison function, not by any inherent "front/back" property.

```
index.html JS indexjs X
JS indexjs > main > render
9 function main() {
381   const MODE_NAMES = {
382     [gl.TRIANGLES]: "gl.TRIANGLES",
383     [gl.LINE_LOOP]: "gl.LINE_LOOP",
384     [gl.LINES]: "gl.LINES",
385     [gl.LINE_STRIP]: "gl.LINE_STRIP",
386     [gl.POINTS]: "gl.POINTS",
387     [gl.TRIANGLE_STRIP]: "gl.TRIANGLE_STRIP",
388   };
389   const state = {
390     mode: gl.TRIANGLES,
391     depth: true,
392     cubeFirst: true,
393   };
394
395   /*===== Draw the points to the screen =====*/
396   function render() {
397     // Clear
398     gl.clearColor(0.13, 0.13, 0.15, 1.0);
399     if (state.depth) {
400       gl.enable(gl.DEPTH_TEST);
401       gl.depthFunc(gl.GREATER);
402       gl.clearDepth(0.0);
403       gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
404     } else {
405       gl.disable(gl.DEPTH_TEST);
406       gl.clear(gl.COLOR_BUFFER_BIT);
407     }
408
409     const mode = state.mode;
410     const cube = { first: 0, count: CUBE_VERT_COUNT };
411     const stair = { first: CUBE_VERT_COUNT, count: STAIR_VERT_COUNT };
412
413     if (state.cubeFirst) {
414       gl.drawArrays(mode, cube.first, cube.count);
415       gl.drawArrays(mode, stair.first, stair.count);
416     } else {
417       gl.drawArrays(mode, stair.first, stair.count);
418       gl.drawArrays(mode, cube.first, cube.count);
419     }
420
421     // Update status label
422     const orderStr = state.cubeFirst ? "Cube → Staircase" : "Staircase → Cube";
423     document.getElementById("status").textContent =
424       `ID: 240014 | Alinur S.\n` +
425       `Mode: ${MODE_NAMES[mode]}\n` +
426       `Depth: ${state.depth ? "ON" : "OFF"}\n` +
427       `Order: ${orderStr}\n` +
428       `[1-6] mode [D] depth [S] swap order`;
429   }
430
431   // Keyboard handler
```



Screenshot E2-B: gl.GREATER + clearDepth(0.0) - back faces visible, front faces hidden (inverted depth)

E3 - Drawing Modes on Triangle Data (4pts)

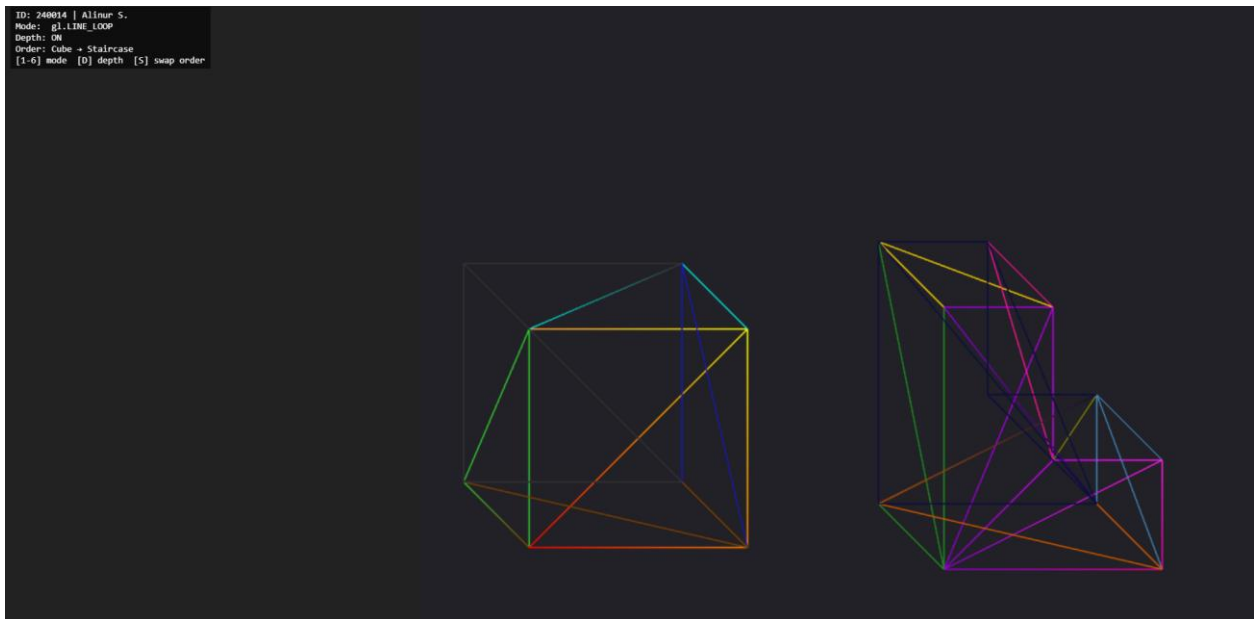
Data context

The cube's 36 vertices are stored as 12 groups of 3 (one triangle each), ordered by face: front (2 tris), back (2), top (2), bottom (2), left (2), right (2). This ordering is optimised for gl.TRIANGLES - it is not a strip or a loop sequence.

Mode 2 - gl.LINE_LOOP

LINE_LOOP connects vertex $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 35$, then closes back from $35 \rightarrow 0$. It draws one continuous closed polyline through all 36 vertices in the order they appear in the buffer.

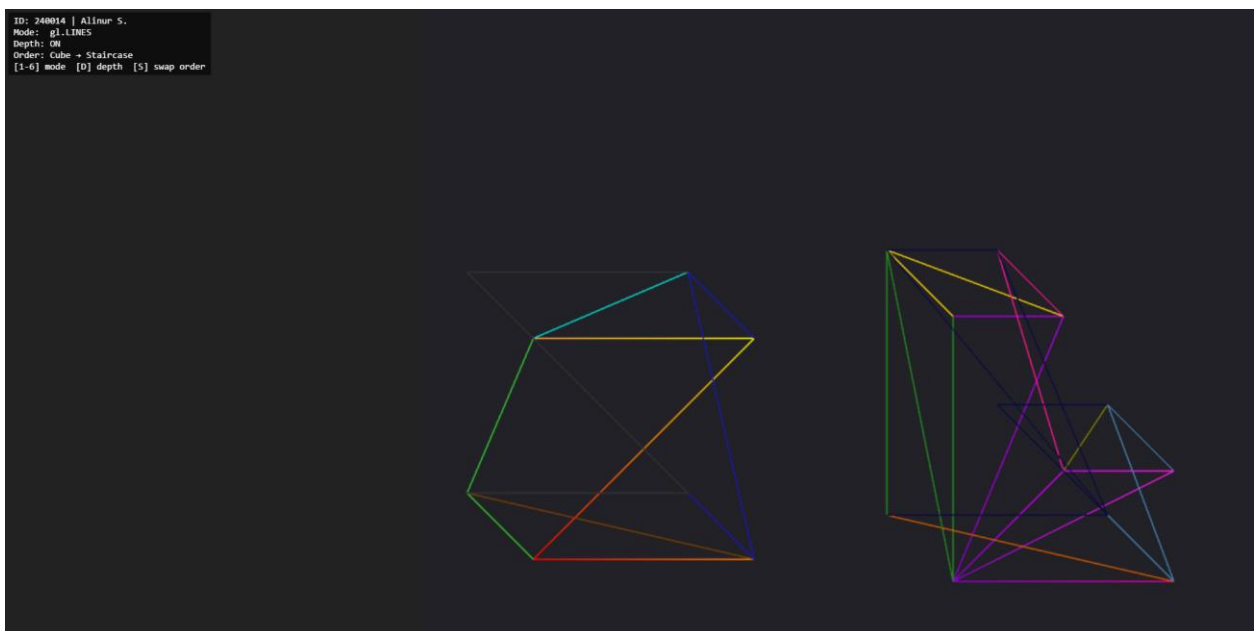
Result: Instead of clean box edges, LINE_LOOP draws diagonal lines across faces. This happens because the vertices are interleaved for triangle construction, not edge construction. For example, the two triangles of the front face are stored as: BL, BR, TR, BL, TR, TL. LINE_LOOP connects them in that order: $BL \rightarrow BR \rightarrow TR \rightarrow BL \rightarrow TR \rightarrow TL$, then continues into the back face vertices. The repeated BL creates the "diagonal" — the line goes $BR \rightarrow TR$ then snaps back to BL before continuing to TR again, drawing an interior diagonal of the face. The extra line is the closing edge from vertex 35 back to vertex 0, which crosses across the entire shape.



Screenshot E3-A: Mode 2 (`gl.LINE_LOOP`) on the cube - diagonals visible, unexpected closing edge

Mode 3 - `gl.LINES`

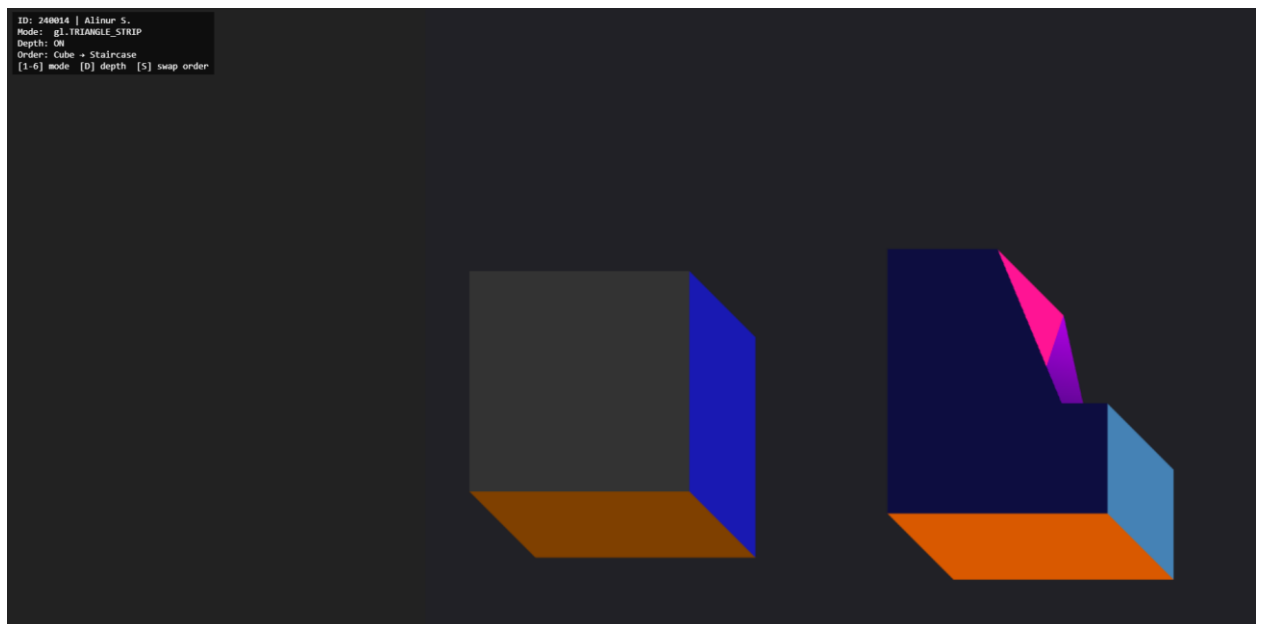
`LINES` treats every consecutive pair of vertices as an independent line segment: (0,1), (2,3), (4,5), ... (34,35). For the cube this produces 18 line segments. Each triangle contributes one segment (its first edge) because the triangle's first two vertices form a pair, and the third vertex pairs with the next triangle's first vertex — but that next pair spans two different faces. The result is a partial wireframe: only some edges of each face are drawn (specifically the base edge of each triangle), and many edges are missing entirely. It looks like an incomplete, asymmetric wireframe.



Screenshot E3-B: Mode 3 (`gl.LINES`) on the cube — partial edges only, asymmetric wireframe

Mode 6 - gl.TRIANGLE_STRIP

TRIANGLE_STRIP reuses the last two vertices for each new triangle: triangle k uses vertices $(k, k+1, k+2)$. This means triangle 0 is (v_0, v_1, v_2) , triangle 1 is (v_1, v_2, v_3) , triangle 2 is (v_2, v_3, v_4) , and so on — completely ignoring the intended grouping of three consecutive vertices per triangle. Because our data is arranged for TRIANGLES (groups of 3), TRIANGLE_STRIP consumes them in a sliding window instead, producing wildly incorrect geometry. Vertices from different faces get combined into the same triangle, creating long stretched "fan" triangles that cross the screen. The result looks like a distorted web of overlapping polygons bearing no resemblance to the original cube.



Screenshot E3-C: Mode 6 (gl.TRIANGLE_STRIP) on triangle-ordered data - distorted, incorrect geometry

E4 - The Missing Bind (4pts)

Setup

In Ch.3 Part 2, the `gl.vertexAttribPointer` for `aPosition` is called **after** the colour buffer has been bound to `index.js` :

```
// Incorrect order (reproducing Ch.3 Part 2 mistake):
gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(positions), gl.STATIC_DRAW);

gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);          // ← colour bound now
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(colors), gl.STATIC_DRAW);

// aPosition pointer set while colorBuffer is still bound:
const posLoc = gl.getAttribLocation(program, "aPosition");
gl.vertexAttribPointer(posLoc, 3, gl.FLOAT, false, 0, 0); // ← reads
colorBuffer!
gl.enableVertexAttribArray(posLoc);

// Colour pointer also set from colorBuffer:
const colLoc = gl.getAttribLocation(program, "aVertexColor");
gl.vertexAttribPointer(colLoc, 4, gl.FLOAT, false, 0, 0);
gl.enableVertexAttribArray(colLoc);
```

Prediction

gl.vertexAttribPointer does not read data immediately; it records a reference to whichever buffer is currently bound to the call. If colorBuffer is bound when gl.ARRAY_BUFFER at the moment of aPosition's pointer is set, then aPosition will read its xyz coordinates from the colour buffer's RGBA float data. The colour buffer holds values like (1.0, 0.0, 0.0, 1.0), (1.0, 0.5, 0.0, 1.0), etc. The vertex shader will interpret these as clip-space (x, y, z) coordinates (with w=1 default for a vec4 attribute). All colour values are between 0 and 1, so all "position" values will be in the top-right quadrant of clip space. The shapes will appear compressed into a small patch in the upper-right corner of the canvas, coloured by whatever the colour attribute pointer reads.

Result: Both the cube and staircase appeared as distorted blobs crammed into the upper-right corner of the canvas (clip coordinates 0–1 in both x and y). The colours were scrambled - the colour attribute was also reading from the colour buffer but at an offset dictated by the float layout, producing incorrect colour assignments.



```
ID: 240014 | Alinear 5.  
Mode: gl.TRIANGLES  
Depth: ON  
Order: Cube + Staircase  
[1-0] mode [0] depth [5] swap order
```

Screenshot E4-A: Missing bind - both shapes squashed into the top-right quadrant, colours garbled

Why no WebGL error?

WebGL does not validate semantic correctness - it only checks that the buffer exists, the type is valid, and the stride/offset are within bounds. Pointing aPosition at colorBuffer is syntactically legal: both are ARRAY_BUFFER objects containing 32-bit floats. WebGL has no concept of "this buffer was intended for colour data." It simply reads floats from the recorded buffer address and supplies them to the shader. No error is reported because no rule was violated at the API level - only the programmer's intent was violated.

Fix: Always bind the correct buffer immediately before each `gl.vertexAttribPointer` call, as done in my final `index.js` :

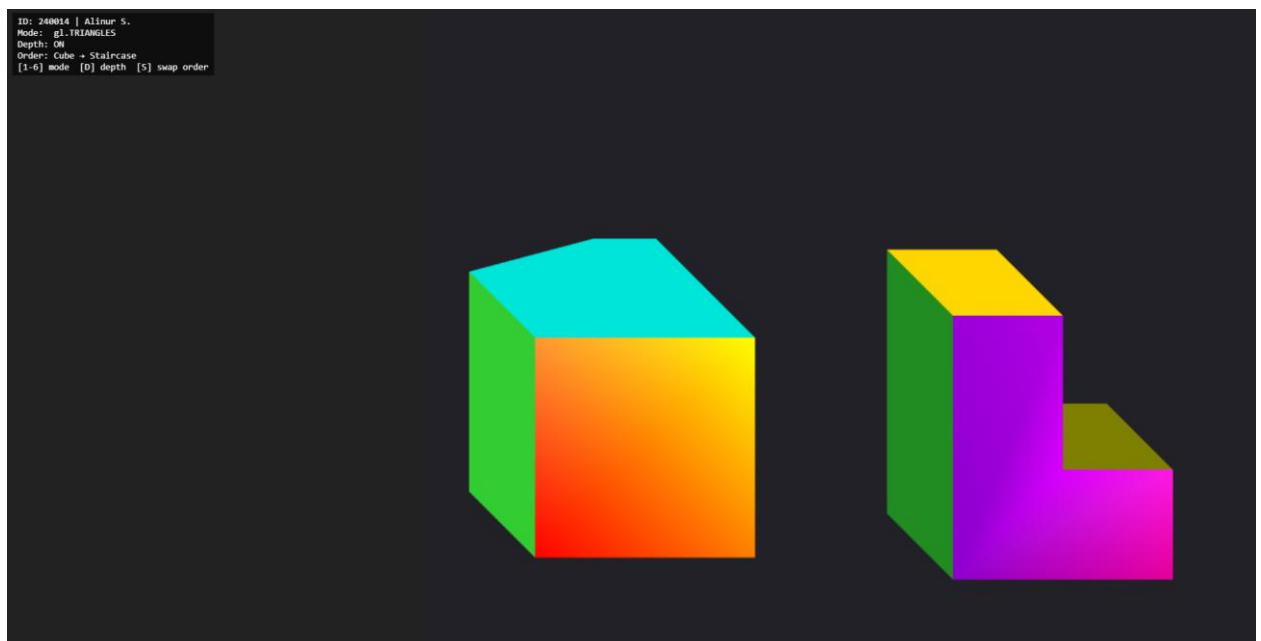
```
gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer); // ← correct bind  
gl.vertexAttribPointer(posLoc, 3, gl.FLOAT, false, 0, 0);  
gl.enableVertexAttribArray(posLoc);  
  
gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);      // ← correct bind  
gl.vertexAttribPointer(colLoc, 4, gl.FLOAT, false, 0, 0);  
gl.enableVertexAttribArray(colLoc);
```

E5 - Clipping (2pts)

Test (a): Move one cube vertex to $z = 1.2$

Clip space z range is $[-1, +1]$. A vertex at $z = 1.2$ is beyond the far plane. WebGL clips any triangle with a vertex outside clip space. The triangle that contains this vertex will be partially clipped: the portion of the triangle that still falls within the clip volume remains visible; the portion beyond $z = 1$ is cut off. We will see a truncated face - the face appears to end abruptly as if sliced by an invisible plane.

The back-right vertex of the cube was moved to $z = 1.2$. The two triangles of the right face that use this vertex were clipped: they appeared truncated, with the portion beyond $z = 1.0$ invisible. The right face now had a clean diagonal cut where the $z = 1.0$ far plane intersected it. No error was reported; WebGL clips silently.

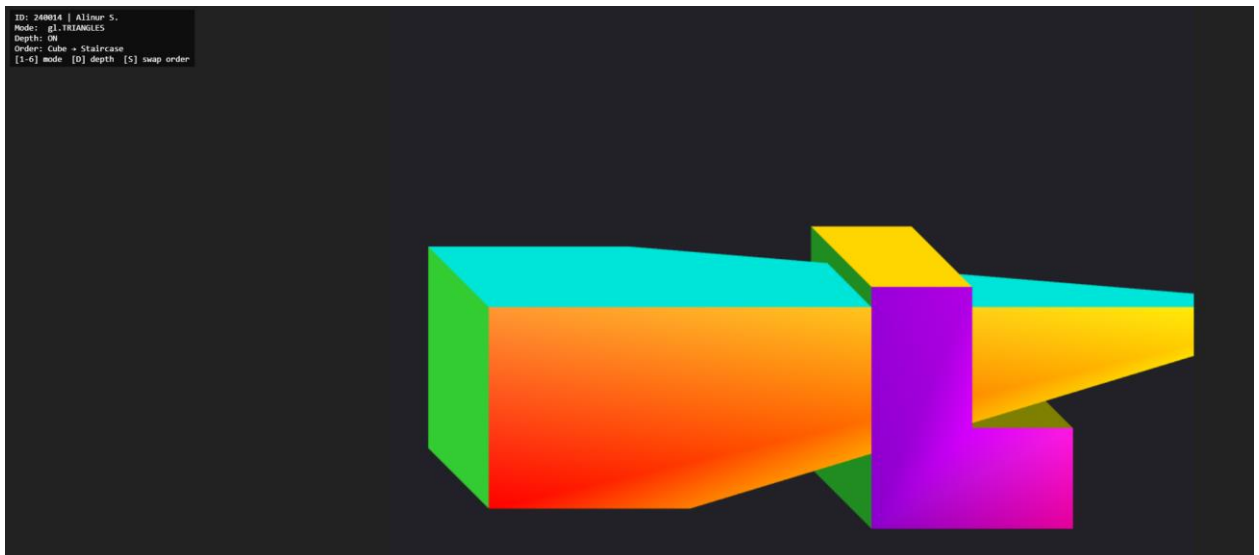


Screenshot E5-A: One cube vertex at $z = 1.2$ — right face clipped at the far plane, triangular slice missing

Test (b): Move one cube vertex to $x = 1.4$

Clip space x range is $[-1, +1]$. A vertex at $x = 1.4$ is past the right clipping plane. Triangles using this vertex are clipped on the right side of the canvas. The affected faces will appear to be cut off at $x = 1.0$ - the right portion of those triangles is invisible.

The front-top-right cube vertex was moved to $x = 1.4$. The front and right faces using that vertex were clipped at $x = 1.0$. The front face became a trapezoid (its right portion cut off), and the right face was almost entirely removed. Again, WebGL clipped silently - no error, no warning. The geometry beyond $x = 1.0$ simply does not exist on screen.



Screenshot E5-B: One cube vertex at $x = 1.4$ — front and top faces clipped at the right edge of the viewport

E6 - Buffer Sizes (2 pts)

Vertex counts

Cube: 36 vertices (6 faces $\times$ 2 triangles $\times$ 3 vertices)

Two-step staircase: 60 vertices (front L-face: 12, back L-face: 12, 6 side rects $\times$ 6: 36)

$N = 96$ vertices total

Buffer size calculation

Position buffer: each vertex has 3 floats (x, y, z) $\times$ 4 bytes = 12 bytes/ vertex

$96 \times 3 \times 4 = 1152$ bytes

Colour buffer: each vertex has 4 floats (r, g, b, a) $\times$ 4 bytes = 16 bytes/ vertex

$96 \times 4 \times 4 = 1536$ bytes

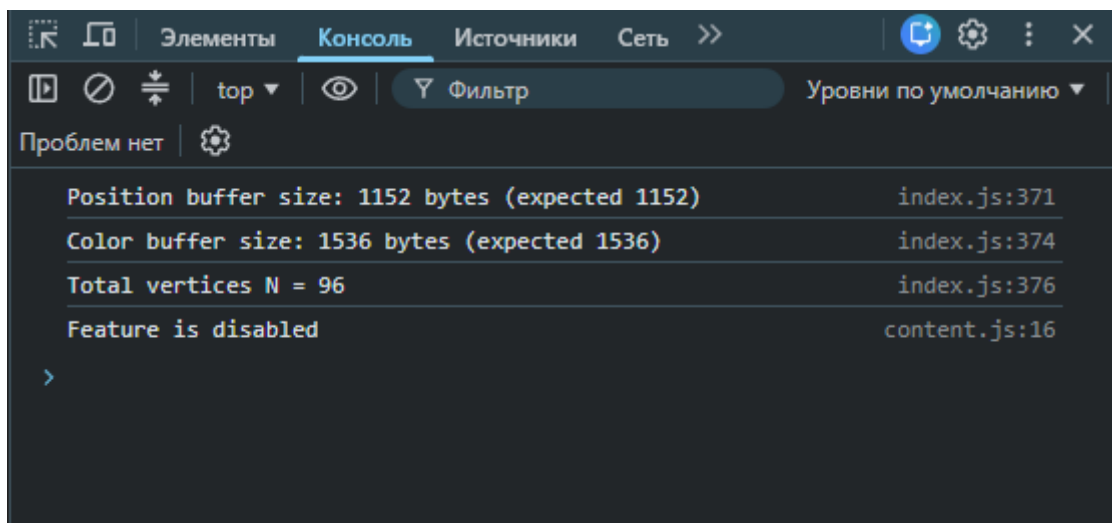
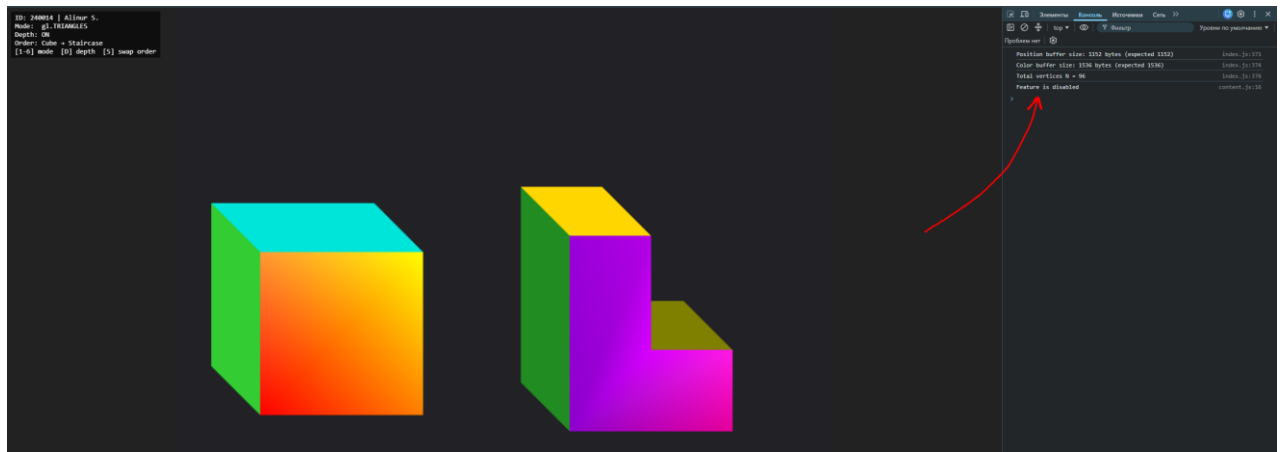
Verification with `gl.getBufferParameter`

The following was added to `index.js` after buffer initialisation:

```
gl.bindBuffer(gl.ARRAY_BUFFER, buffers.position);
console.log(`Position buffer size: ${gl.getBufferParameter(gl.ARRAY_BUFFER,
gl.BUFFER_SIZE)} bytes`);
// Output: Position buffer size: 1152 bytes

gl.bindBuffer(gl.ARRAY_BUFFER, buffers.color);
console.log(`Color buffer size: ${gl.getBufferParameter(gl.ARRAY_BUFFER,
gl.BUFFER_SIZE)} bytes`);
// Output: Color buffer size: 1536 bytes

console.log(`Total vertices N = ${TOTAL_VERTS}`);
// Output: Total vertices N = 96
```



Screenshot E6-A: Browser console output showing buffer sizes: position 1152 bytes, colour 1536 bytes, $N = 96$

Both computed values match the measured values. The console.assert checks in index.js also confirm: the colour array has exactly 4 values per vertex for each sub-array, preventing silent length mismatches.

Development Log

Bug: The two-step staircase had several vertices at drawn $x = 0$ (the canvas centre) after applying the depth-illusion back-face shift of -0.15 . The left edge of the raw profile was at $x = 0.15$; after shifting, $x_{\text{back}} = 0.15 - 0.15 = 0.00$ - exactly on the boundary, not strictly in the right half as required.

Found by: Added a Node.js verification script that iterated over all stair vertices after shift and checked $\text{stair}[i] > 0$ for every x -component. The check printed BAD stair $x: 0$ at vert 13 (and several others), pinpointing the problem immediately.

Fix: Shifted the entire staircase profile 0.05 units to the right (left edge: $0.15 \rightarrow 0.20$). After this change, the back-face leftmost $x = 0.20 - 0.15 = 0.05 > 0$, satisfying the constraint. Re-ran the verification script - all checks passed.